

GENERACIÓN DE CERTIFICADOS PARA HTTPS

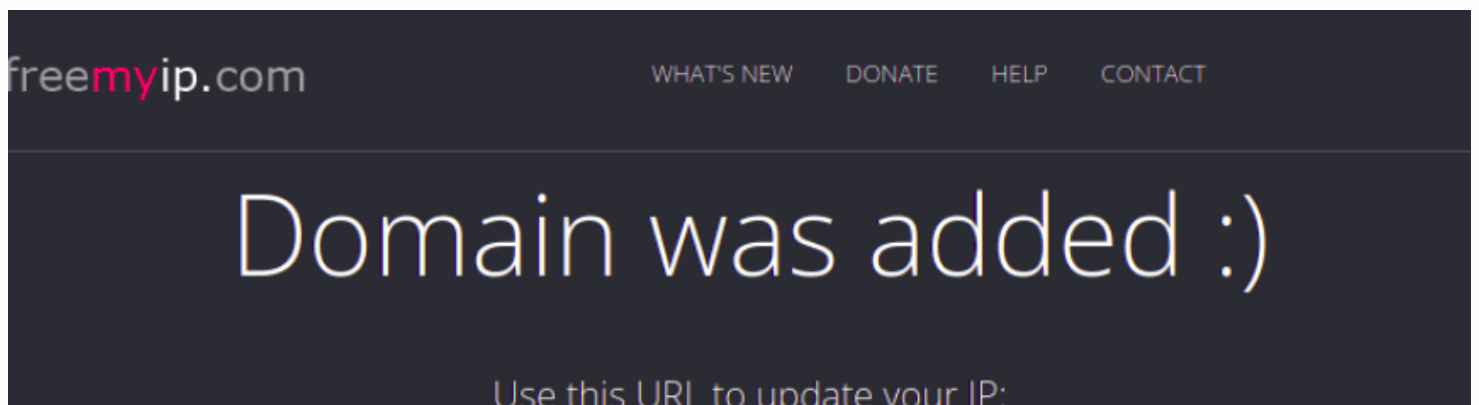
Si quieres generar certificados válidos reales, necesitas un dominio (visita freemyip.com y crea uno) y un lugar físico donde albergar tu web (o en la nube con AWS)

Si lo haces desde un lugar físico como tu casa, esto es muy sencillo, basta redirigir el puerto 80 de tu router a la ip de tu ordenador donde tu web esta funcionando. A lo largo del documento yo generaré un certificado para una Web hosteada tras mi router, es decir, una web que corre en un servidor de mi casa. Si lo hacéis desde AWS los pasos son similares pero tenéis que tener una web disponible en AWS y no hará falta hacer redirección de puertos del router al servidor (ordenador que sirve la web).

Para la generación de un certificado válido, los pasos que debes seguir serán los siguientes (recuerda que mi dominio será: clasesiaw.freemyip.com y tu debes elegir el tuyo propio y sustituir clasesiaw.freemyip.com por el tuyo en este proceso).

PASO 1

Solicita un dominio gratuito en [freemyip](https://freemyip.com). Yo he solicitado [clasesIAW.freemyip.com](https://clasesiaw.freemyip.com)



Esta página te dará un enlace que debes ejecutar para asociar la IP de tu router al dominio en concreto, es decir, para que cuando una persona quiera entrar a tu página web esa página esté asociada a la IP de tu router.

Como la IP de tu router puede cambiar tras reinicios, recomiendo ejecutar esta url en crontab cada 5 minutos por si la ip de tu router cambia y que así no se caiga tu web.

PASO 2

Debes crear un `docker-compose.yml` con todas las dependencias de tu proyecto. En la mía (a la que hicistéis un clon en github) el siguiente `docker-compose.yml` sirve.

```
services:
  node:
    build: .
    restart: always
    expose:
      - "5050"
    depends_on:
      - mongo
    networks:
      - mi-app-node
    environment:
      - PATH_DB=mongo
      - SSL=false
    deploy:
      replicas: 3
      restart_policy:
        condition: always
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5050"]
      interval: 30s
      retries: 3
      start_period: 5s
      timeout: 10s
    volumes:
      - imagenes_data:/imagenes
```

```
mongo:
  image: mongo:8.0
  container_name: contenedor_mongodb
  ports:
    - "27019:27017"
  networks:
    - mi-app-node
  volumes:
    - mongo_data:/data/db
```

```
nginx:
  image: nginx:latest
  container_name: contenedor_nginx
  ports:
    - "80:80"
  depends_on:
    - node
  networks:
    - mi-app-node
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - ./certbot/www:/var/www/certbot
```

```
networks:
  mi-app-node:
    driver: bridge
```

```
volumes:
  mongo_data:
  imagenes_data:
```

Fijaos que en realidad nginx no es una dependencia del proyecto, pero como ya he comentado en clase, lo usamos como proxy inverso, es decir, cuando alguien quiera hacer una peti a mi servidor web (que es NODE), antes pasará por nginx. De hecho, nginx nos ayudará a generar los certificados necesarios tal y como veréis posteriormente.

PASO 3

Después cambiamos el archivo nginx.conf. Este archivo define cómo se reenvía nginx a node (el servidor):

```
events {}

http {
    upstream node_backend {
        server node:5050;
    }

    server {
        listen 80;
        server_name clasesiaw.freemyip.com;

        location /.well-known/acme-challenge/ {
            root /var/www/certbot;
        }

        location / {
            proxy_pass http://node_backend;
        }
    }
}
```

Esta config de nginx es clásica. No entro en detalles pero podéis buscar vosotros información al respecto.

PASO 4

Después haz una redirección de puertos de tu router hacia el ordenador que sirve nginx, para que corra al servicio :

▼ Port Forwarding

[¿Qué se debe tener en cuenta al configurar el reenvío de puertos?](#)

▶ cable	☐ Activado ☒ Desactivado	
▼ http	☒ Activado ☐ Desactivado	

Nombre	http
Protocolo	TCP
Dirección IP del host WAN	0 0 0 0 ~ 0 0 0 0
LAN Host	192.168.1.145
Puerto WAN	80 ~ 80
Puerto de host LAN	80 ~ 80

Crear elemento nuevo

O hostea en AWS (tú eliges pero si usas AWS búscate la habichuelas)

Sobre tu proyecto de tu página web, crea:

```
mkdir certbot/www
mkdir certbot/conf
```

será fundamental para el volumen montado en docker-compose.

PASO 4

Después: una vez tu página web esté funcionando (es decir, hayas hecho docker compose up), ejecuta lo siguiente en terminal sobre la carpeta de tu proyecto (creas un contenedor temporal que muere al finalizar). Este contenedor que levanta no se crea a partir del docker compose anterior, es completamente independiente!!:

```
docker run --rm \
-v $(pwd)/certbot/www:/var/www/certbot \
-v $(pwd)/certbot/conf:/etc/letsencrypt \
certbot/certbot certonly \
--webroot \
-w /var/www/certbot \
-d clasesiaw.freemyip.com \
--agree-tos \
--no-eff-email \
-m tuemail@dominio.com
```

Recuerda cambiar el nombre del dominio. Este comando se usa para pedir un certificados HTTPS públicos y privados a Let's Encrypt para tu dominio. Internamente estás levantando un contenedor con imagen de Certbot y realizas tres acciones fundamentales.

PASO 4.1

1- En primer lugar, genera el par de claves del servidor:

```
certbot/conf/live/clasesiaw.freemyip.com/fullchain.pem.
```

```
certbot/conf/live/clasesiaw.freemyip.com/privkey.pem
```

es decir, crea una clave privada, que permanecerá almacenada de forma local y secreta en tu server, y una clave pública, que identificará al servidor. A continuación, Certbot crea una petición de certificado que incluye clave pública del servidor y el nombre del dominio solicitado, en este caso `clasesiaw.freemyip.com`. Finalmente, dicha petición se envía a Let's Encrypt, que actuará como Autoridad de Certificación.

PASO 4.2

2- Let's Encrypt accede por Internet a una URL concreta dentro de mi dominio (`/.well-known/acme-challenge/`). (que ha sido creada por el comando anterior y se mete en un volumen `/var/www/certboot`). Si puede acceder correctamente a dicha ruta, Let's Encrypt concluye que quien controla ese servidor controla también el dominio, y da por validada la solicitud. Si todo ha ido bien obtendrás al ejecutar lo anterior obtendrás:

```
nanolopi5@nanolopi5:~/Project_Node$ sudo nano docker-compose.yml
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for clasesiaw.freemyip.com

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/clasesiaw.freemyip.com/fullchain.pem
Key is saved at: /etc/letsencrypt/live/clasesiaw.freemyip.com/privkey.pem
This certificate expires on 2026-04-09.
These files will be updated when the certificate renews.
NEXT STEPS:
- The certificate will need to be renewed before it expires. Certbot can automatically renew the certificate in the background, but you may need to take steps
  to enable that functionality. See https://certbot.org/renewal-setup for instructions.

-----
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
 * Donating to EFF: https://eff.org/donate-le
-----
nanolopi5@nanolopi5:~/Project_Node$ sudo nano docker-compose.yml
nanolopi5@nanolopi5:~/Project_Node$ sudo cat docker-compose.yml
version: "3.9"
```

PASO 4.3

3-Por último el comando anterior también hace que Let's Encrypt firme el certificado público del servidor (fullchain.pem). El resultado de todo esto es un certificado válido públicamente, acompañado de la cadena de certificados intermedios, todo ello incluido en el archivo:

```
certbot/conf/live/clasesiaw.freemyip.com/fullchain.pem.
```

PASO 4.4

4-Una vez conseguido lo anterior, necesitas pasos opcionales para la funcionalidad de tu web, entre ellos, hacerle un docker compose down y volver a editar este nginx.conf ya que tenemos que configurar https (antes sólo teníamos http en el servidor).

```
events {}

http {

    upstream node_backend {
        server node:5050;
    }

    server {
        listen 80;
        server_name clasesiaw.freemyip.com;

        location /.well-known/acme-challenge/ {
            root /var/www/certbot;
        }

        location / {
            return 301 https://$host$request_uri;
        }
    }

    server {
        listen 443 ssl;
        server_name clasesiaw.freemyip.com;

        ssl_certificate      /etc/letsencrypt/live/clasesiaw.freemyip.com/fullchain.pem;
        ssl_certificate_key  /etc/letsencrypt/live/clasesiaw.freemyip.com/privkey.pem;

        location / {
            proxy_pass http://node_backend;
        }
    }
}
```

y cambiar el contenedor de nginx para los reenvíos de puertos:

```
nginx:
  image: nginx:latest
  container_name: contenedor_nginx
  ports:
    - "80:80"
    - "443:443"
  depends_on:
    - node
  networks:
    - mi-app-node
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - ./certbot/www:/var/www/certbot
    - ./certbot/conf:/etc/letsencrypt
```

PASO 5

Hecho esto, cuando un usuario cualquiera de INTERNET accede posteriormente a

<https://clasesiaw.freemyip.com>

el navegador debe comprobar que el certificado presentado por el servidor no es autofirmado, sino que ha sido emitido por una Autoridad de Certificación de confianza.

Durante el inicio de la conexión, el servidor nginx envía su certificado público al navegador. El navegador valida en primer lugar que dicho certificado esté firmado por una entidad de confianza (ya lo está). Para ello, extrae del certificado la clave pública del servidor y la firma digital de Let's Encrypt, y busca en su almacén de confianza el certificado raíz de Let's Encrypt. Utilizando la clave pública de esa CA, el navegador verifica la firma del certificado del servidor. Si la verificación es correcta, el certificado se considera válido, el servidor se considera auténtico y puede continuar el establecimiento de la conexión TLS de forma segura.

Y ya podrás acceder por https a clasesiaw.freemyip.com (el nombre del dominio que cree en freemyip) !



**¡Bienvenido
a
la
Formación
Profesional!**

Unete al nuevo ClassRoom

Solo a un click

▶ Iniciar sesion

👤 Crear usuario

¿Has olvidado la contraseña?

🔒 Ponte en contacto con el
administrador

⚠ Esta pagina solo está realizada para el
acceso a alumnos del IES Romero Vargas